SOFTWARE ENGINEERING ACADEMIC PROJECT — MID-SEMESTER EVALUATION

Department of Computer Science & Engineering

Requirements Model: Use Cases & User Stories

Project: AutoTestHub (Web Automation & Testing Platform)

USE CASE SPECIFICATIONS • USER STORIES • GHERKIN CRITERIA

SPECIFICATION DETAILS

Artifact: Requirements Model Document

Evaluation Milestone: Mid-Semester Review

Primary Actors: QA Engineer, Software Developer, System Pipeline

Document ID: ATH-REQ-MOD-2026

DEVELOPMENT TEAM

Shristi Neekhra (Integration Lead)

Shifa Siddiqui (Backend Architect)

Shruti Gupta (Selenium Engine Architect)

Shriya Agarwal (Frontend Specialist)

Academic Guide: [Supervisor Name]

1. System Use Case Model

The AutoTestHub requirements model defines interactions between external human actors (QA Engineers, Developers), secondary automation actors (System CI/CD pipelines), and the core AutoTestHub platform boundary.

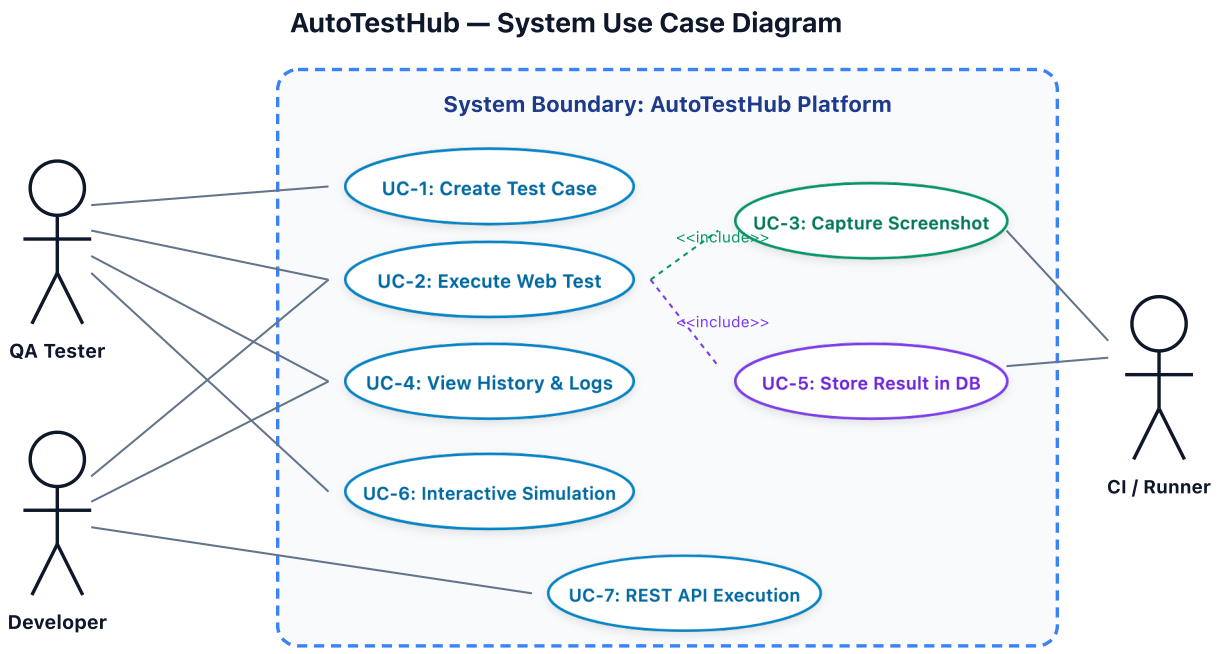


Figure 1: AutoTestHub System Use Case Diagram illustrating primary and secondary actor boundaries.

1.1 Actor Definitions

Actor Name	Actor Classification	Responsibilities & Goals
QA Tester	Primary Human Actor	Configures automated test definitions, executes browser tests, audits visual screenshots, and monitors regressions.
Software Developer	Primary Human Actor	Invokes test execution via REST APIs during local feature development; runs simulation mode on the Control Desk.
CI / System Runner	Secondary System Actor	Automated background runner that executes test pipelines and polls API health status.

2. Detailed Use Case Specifications

2.1 UC-01: Create Automated Test Case

Use Case ID / Name	**UC-01: Create Automated Test Case**
Primary Actor	QA Tester / Software Developer
Preconditions	The Express backend server is running and connected to MongoDB.
Trigger	Actor submits a test definition via REST client or UI form.
Main Success Scenario	1. Actor sends a `POST /api/tests` request with test name, target URL, and actions array. 2. System validates the payload schema (verifying URL scheme and non-empty actions). 3. System instantiates a new `TestCase` Mongoose model and persists it to MongoDB. 4. System returns HTTP 201 Created with the generated document ID and timestamps.
Extensions / Alternate Flows	**2a. Validation Failure:** If required fields are missing, system returns HTTP 400 Bad Request with an error message detailing the missing schema properties.
Postconditions	A new test case document is persistently stored in MongoDB and ready for execution.

2.2 UC-02: Execute Web Test via Integration Pipeline

Use Case ID / Name	**UC-02: Execute Web Test via Integration Pipeline**
Primary Actor	QA Tester / Developer / System Runner
Preconditions	A valid `testCaseId` exists in the database; Chrome browser is installed.
Trigger	Actor issues `POST /api/tests/:id/run`.
Main Success Scenario	1. Express API route receives execution request and invokes `runTestExecution(testCase)`. 2. Execution Service spawns `python3 runner.py` as a child process. 3. Node.js serializes test definition JSON and pipes it directly into Python's `stdin`. 4. Python runner initializes Chrome via Selenium Manager, executes actions, and asserts title. 5. Python captures a PNG screenshot, outputs the single-line JSON result to `stdout`, and exits (code 0). 6. Execution Service parses the JSON string, saves a new `TestResult` record in MongoDB, and returns HTTP 200 OK.
Extensions / Alternate Flows	**4a. Element Timeout:** If an element is missing, Python catches TimeoutException, captures a failure screenshot, emits FAILED JSON (code 1), and Node.js persists the failure record.
Postconditions	A timestamped execution result and screenshot are permanently recorded.

2.3 UC-03: Interactive Control Desk Simulation & Real Run

Use Case ID / Name	**UC-03: Interactive Control Desk Run**
Primary Actor	QA Tester / Evaluator

Preconditions	User opens `http://localhost:5000` in a modern web browser.
Trigger	User selects an automation task (Demo, Search, Form, Multi-Submit) and clicks "Run task".
Main Success Scenario	1. Frontend UI collects user parameters (URL, search query, or form data) and toggle states (speed, headless). 2. If connected to live backend, sends `POST /api/run`; otherwise activates client simulation mode. 3. Real Chrome window opens (or simulation moves step-by-step in viewport). 4. Real-time terminal output logs each executed WebDriver statement. 5. Progress bar reaches 100%, screenshot is saved, and run history item is appended.
Postconditions	User visualizes automated browser interactions in real-time with comprehensive logs.

3. User Stories & Acceptance Criteria

User stories represent agile requirement definitions structured in the industry-standard *As a / I want / So that* format, accompanied by formal *Given-When-Then* (Gherkin) acceptance criteria.

User Story 1: Declarative Test Definition Ingestion

> **As a QA Automation Engineer**,
> **I want** to define browser test scenarios using structured JSON files,
> **So that** I can author robust regression tests without writing boilerplate WebDriver code.

```
Scenario: Successfully validate and ingest a valid JSON test case
  Given a test definition containing a valid "testCaseId", "url", and supported "actions"
  When the runner processes the JSON file or stdin stream
  Then the validation resolver must return true with no error messages
  And execution proceeds to browser initialization.

Scenario: Reject an insecure or malformed URL
  Given a test definition containing a URL with scheme "file:///etc/passwd"
  When the URL validator checks the string
  Then a ValueError must be raised immediately before launching Chrome
  And a descriptive security error message must be returned.
```

User Story 2: Explicit Dynamic Action Synchronization

> **As a Software Developer**,
> **I want** the test engine to wait explicitly for elements to become visible and clickable,
> **So that** test executions do not suffer from asynchronous timing flakiness.

```
Scenario: Synchronize text input on dynamically rendered form element
  Given a web page with an input element identified by name "my-text"
  When the runner dispatches a "TYPE" action with value "Automation User"
  Then WebDriverWait must wait up to the configured timeout for element visibility
  And the existing text must be cleared prior to typing new keystrokes.
```

User Story 3: Automated Evidence Capture & Auditing

> **As an Engineering Lead**,
> **I want** the system to automatically save timestamped screenshots on both passing and failing runs,
> **So that** our team has incontrovertible visual evidence of UI states during regression cycles.

```
Scenario: Capture screenshot upon assertion verification
  Given an ongoing test run executing an "ASSERT_TITLE" check
  When the page title matches the expected string
  Then the browser manager must write a PNG screenshot to the designated artifacts directory
  And the relative file path must be embedded in the result JSON payload.
```

User Story 4: Process Cleanup & Resource Safeguard

> **As a DevOps Engineer**,
> **I want** the Python worker to guarantee browser process termination upon failure,

> **So that** unhandled exceptions do not leak orphaned ChromeDriver instances on host servers.

```
Scenario: Clean up browser resources after missing element timeout
  Given a test step attempting to click a non-existent button with a 0.5s timeout
  When the timeout expires and an ActionTimeout exception is caught
  Then a failure screenshot must be captured
  And driver.quit() must be executed in the finally block
  And subsequent test executions must run in a clean, isolated session without port conflicts.
```